In this notebook, we will introduce PyTorch, talk about its important concepts and features, and eventually train an MNIST classifier using what we have learned.

6.8300 / 6.8301 PyTorch Tutorial (02/28/2023, 03/2/2023)

Written by Tongzhou Wang (MIT)

Taught by Sai Bangaru (MIT), Victor Rong (MIT)

推荐:

- 使用个人带英伟达显卡的笔记本，按照教程安装支持cuda的pytorch
- 使用Kaggle[https://www.kaggle.com/]的免费GPU资源(每周30小时T4x2/P100) (https://www.kaggle.com/%5D%E7%9A%84%E5%85%8D%E8%B4%B9GPU%E8%B5%84%E6%BA%9C
  - 登录后点击左上角create new notebook
  - 点击左上角Settings-> Accelerator-> GPU
  - 国内网络可以访问，但速度较慢，建议科学上网

什么是 PyTorch?

1. GPU加速的张量运算库（类似NumPy，但速度更快）
2. 基于动态计算图的可微编程
3. 灵活且高效的**神经网络**库
4. 以Python为优先的框架（易于与其他Python库集成、调试和扩展）
   - 可以快速地与NumPy数组相互转换，能与其他Python库集成。
   - 可使用你喜欢的Python调试器。
   - 可以通过Python/C++扩展添加自定义操作。
   - 能够使用C++ API在纯C++环境中运行。

Useful links:

- PyTorch 文档：https://pytorch.org/docs/stable/index.html (https://pytorch.org/docs/stable/index.html) 大多数数学运算都能以 `torch.*` 或 `Tensor.*` 的形式找到。
- PyTorch 官方教程（60 分钟闪电战是个不错的入门选择）：https://pytorch.org/tutorials/ (https://pytorch.org/tutorials/)
  - 迁移学习教程：https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html (https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html)
- PyTorch 示例（深度卷积生成对抗网络、ImageNet 训练、强化学习等）：https://github.com/pytorch/examples/ (https://github.com/pytorch/examples/)

1. GPU加速的张量运算库

A Tensor is a multi-dimensional array (matrix).

In [1]:
```
import torch
```

In [2]:
```
# Create a 3x5 matrix filled with zeros

x = torch.zeros(3, 5)
print(x)
```

```
tensor([[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]])
```

In [3]:
```
# Create a 3x5 matrix filled with random values

y = torch.randn(3, 5)
print(y)
```

```
tensor([[-1.1980,  0.0130, -0.5197, -0.2960, -1.1673],
        [-0.4970, -0.0252, -0.0917,  0.1555, -0.5487],
        [-0.9453,  0.0076, -0.0353,  0.0560,  1.1244]])
```

In [4]:
```
# Shape manipulations

print('\n.t()  (transpose): ')
print(y.t())

print('.reshape(5, 3): ')
print(y.reshape(5, 3))

# 注意：使用 .view() 和 .reshape() 时可能会有内存方面的考虑
# 你可以查看这个讨论 https://discuss.pytorch.org/t/difference-between-view-reshape-and-permut
# 以获取更多详细信息

# 你可能还会发现其他有用的库，比如 Einops（https://github.com/arogozhnikov/einops）
# 这里有教程：https://einops.rocks/1-einops-basics/
```

```
.t()  (transpose):
tensor([[-1.1980, -0.4970, -0.9453],
        [ 0.0130, -0.0252,  0.0076],
        [-0.5197, -0.0917, -0.0353],
        [-0.2960,  0.1555,  0.0560],
        [-1.1673, -0.5487,  1.1244]])
.reshape(5, 3):
tensor([[-1.1980,  0.0130, -0.5197],
        [-0.2960, -1.1673, -0.4970],
        [-0.0252, -0.0917,  0.1555],
        [-0.5487, -0.9453,  0.0076],
        [-0.0353,  0.0560,  1.1244]])
```

In [5]:
```
print(y)
# Slicing
print(y[1:]) # 选取第一行以及之后的所有行（从0 开始）
# Slicing + select every two elements
print(y[1:, ::2])
```

```
tensor([[-1.1980,  0.0130, -0.5197, -0.2960, -1.1673],
        [-0.4970, -0.0252, -0.0917,  0.1555, -0.5487],
        [-0.9453,  0.0076, -0.0353,  0.0560,  1.1244]])
tensor([[-0.4970, -0.0252, -0.0917,  0.1555, -0.5487],
        [-0.9453,  0.0076, -0.0353,  0.0560,  1.1244]])
tensor([[-0.4970, -0.0917, -0.5487],
        [-0.9453, -0.0353,  1.1244]])
```

In [6]:
```
# Basic arithmetic

print(x + 2)
```

```
tensor([[2., 2., 2., 2., 2.],
        [2., 2., 2., 2., 2.],
        [2., 2., 2., 2., 2.]])
```

In [7]:
```
print(y * (x + 2)) # 逐元素乘法，矩阵乘法是torch.matmul()
```

```
tensor([[-2.3960,  0.0260, -1.0394, -0.5920, -2.3347],
        [-0.9939, -0.0504, -0.1833,  0.3110, -1.0974],
        [-1.8905,  0.0151, -0.0706,  0.1120,  2.2488]])
```

In [8]:
```
print((y * (x + 2)).exp())
```

```
tensor([[0.0911, 1.0264, 0.3537, 0.5532, 0.0968],
        [0.3701, 0.9508, 0.8325, 1.3648, 0.3337],
        [0.1510, 1.0152, 0.9318, 1.1185, 9.4762]])
```

1.1 GUP加速

所有操作都可以在 GPU 上运行。

首先，让我们创建一个表示 GPU 设备的 `torch.device` (https://pytorch.org/docs/stable/tensor_attributes.html#torch-device) 对象。

In [9]:
```
cuda0 = torch.device('cuda:0')  # pick the GPU at index 0
```

In [10]:
```
y
```

Out[10]:
```
tensor([[-1.1980,  0.0130, -0.5197, -0.2960, -1.1673],
        [-0.4970, -0.0252, -0.0917,  0.1555, -0.5487],
        [-0.9453,  0.0076, -0.0353,  0.0560,  1.1244]])
```

In [11]:
```
# 将张量从 CPU 移动到 GPU
# 注意：首次访问 GPU 时，会创建一个context环境，因此这可能需要几秒钟时间。但后续使用会很快。

cuda_y = y.to(cuda0)
print(cuda_y)
```

```
tensor([[-1.1980,  0.0130, -0.5197, -0.2960, -1.1673],
        [-0.4970, -0.0252, -0.0917,  0.1555, -0.5487],
        [-0.9453,  0.0076, -0.0353,  0.0560,  1.1244]], device='cuda:0')
```

In [12]:
```
# Or directly creating a tensor on GPU

cuda_x = torch.zeros(3, 5, device=cuda0)
print(cuda_x)
```

```
tensor([[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]], device='cuda:0')
```

In [13]:
```
# All functions and methods work on GPU tensors

print((cuda_y * (cuda_x + 2)).exp())  # values match the CPU results above!
```

```
tensor([[0.0911, 1.0264, 0.3537, 0.5532, 0.0968],
        [0.3701, 0.9508, 0.8325, 1.3648, 0.3337],
        [0.1510, 1.0152, 0.9318, 1.1185, 9.4762]], device='cuda:0')
```

1.2 NumPy 桥接

将 `torch.Tensor` 转换为 `np.ndarray` 以及反向转换都十分容易。

`torch.Tensor` 和 `np.ndarray` 会共享它们的底层内存位置（前提是 `torch.Tensor` 位于 CPU 上且数据类型 `dtype` 相同），对其中一个进行修改会同时改变另一个。

In [14]:
```
import numpy as np
```

In [15]:
```

x = torch.randn(5)
print(x)
```

```
tensor([-2.0060, -0.8810, -1.5203,  1.4528, -1.4332])
```

In [16]:
```

x.numpy()
```

Out[16]:
```
array([-2.0060022, -0.8810204, -1.5203443,  1.4528455, -1.4331987],
      dtype=float32)
```

In [17]:
```
# Converting a tensor to an array

x = torch.randn(5)
print(x)

# use `my_tensor.numpy()`
x_np = x.numpy()
print(x_np)

# or `np.asarray`

x_np = np.asarray(x)
print(x_np)
```

```
tensor([-0.9183, -0.4228,  0.7674, -0.6369,  1.5479])
[-0.9183419 -0.422774  0.7673733 -0.6368887  1.5478903]
[-0.9183419 -0.422774  0.7673733 -0.6368887  1.5478903]
```

In [18]:
```
# 改变一个，另一个也会改变

x[0] = -1
print(x)
print(x_np)
```

```
tensor([-1.0000, -0.4228,  0.7674, -0.6369,  1.5479])
[-1.        -0.422774  0.7673733 -0.6368887  1.5478903]
```

In [19]:
```
# 转换数组为张量

a = np.random.randn(3, 4)

a_pt = torch.as_tensor(a)
print(a_pt)
```

```
tensor([[ 1.4768, -0.4201, -1.0525,  0.0407],
        [-0.8374, -1.2298,  0.3106, -0.5789],
        [-0.6152,  0.7235, -0.3400,  0.1791]], dtype=torch.float64)
```

In [20]:
```

# CPU tensor和numpy array的内存是共享的
a_pt[0] = -1
print(a)
```

```
[[-1.         -1.         -1.         -1.        ]
 [-0.83739304 -1.22983979  0.31058388 -0.57889854]
 [-0.61516234  0.72353731 -0.34000386  0.17908863]]
```

In [21]:
```
# 但是如果我们改变dtype或device，就会复制
a_half_pt = torch.as_tensor(a, dtype=torch.float16, device=cuda0)
a_half_pt[0] = 9
print(a_half_pt)

print(a)  # original array is not affected
```

```
tensor([[ 9.0000,  9.0000,  9.0000,  9.0000],
        [-0.8374, -1.2295,  0.3105, -0.5791],
        [-0.6152,  0.7236, -0.3401,  0.1791]], device='cuda:0',
       dtype=torch.float16)
[[-1.         -1.         -1.         -1.        ]
 [-0.83739304 -1.22983979  0.31058388 -0.57889854]
 [-0.61516234  0.72353731 -0.34000386  0.17908863]]
```

2. 基于动态计算图的可微编程

基于梯度的优化是当下深度学习热潮中至关重要的一部分。PyTorch采用[反向模式自动求导](https://en.wikipedia.org/wiki/Automatic_differentiation)方法，能够高效地计算对张量所进行的任何运算的梯度。

动态与静态

神经网络本质上是对张量进行的一系列数学运算，这些运算共同构建起一个计算图。

大多数框架，如 TensorFlow、Theano、Caffe 和 CNTK，对计算图持有静态的观点。用户必须先构建好一个神经网络，然后一次又一次地复用相同的结构。若要改变网络的行为方式，就意味着必须从头开始重新构建。

PyTorch采用了一种名为反向模式自动求导的技术，这使得你可以随意改变网络的行为方式，且不会产生任何延迟或额外开销。

动态计算图

当你创建一个 `requires_grad` 标志设置为 `True` 的张量时，[autograd](https://pytorch.org/docs/stable/autograd.html) 引擎会将其视为计算图的一个**叶子**节点。当你使用该张量进行计算时，计算图会动态扩展。当你请求计算梯度时（例如，通过调用 `tensor.backward()` 方法），`autograd` 引擎会沿着计算图反向追溯，并自动为你计算梯度。

In [22]:
```
# Now, we want tensors with `requires_grad=True`

a = torch.ones(3, 5, requires_grad=True)  # tensor of all ones
print(a)  # notice that the `requires_grad` flag is on!
```

```
tensor([[1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1.]], requires_grad=True)
```

In [23]:
```
# Currently `a` has no gradients

print(a.grad)
```

```
None
```

In [24]:
```
# Let's compute the gradient wrt the sum

s = a.sum()
print('sum of a is', s)
```

```
sum of a is tensor(15., grad_fn=<SumBackward0>)
```

In [25]:
```
# 注意 `s` 的 `grad_fn`。它表示用于将梯度从 `s` 传播到计算图中前序节点（在这种情况下是 `a`）

s.backward()  # compute gradient!
print(a.grad)
```

```
tensor([[1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1.]])
```

In [26]:
```
# Yay! Indeed d \sum_a / d a_ij = 1
```

In [26]:
```
# Gradients are automatically **accumulated**

a.sum().backward()
print(a.grad)  # 新梯度会被直接加到之前的梯度上


# 别担心，我们有简单的方法来清除梯度。我们稍后会谈到这些！
```

```
tensor([[2., 2., 2., 2., 2.],
        [2., 2., 2., 2., 2.],
        [2., 2., 2., 2., 2.]])
```

In [27]:
```
a.grad.zero_()
```

Out[27]:
```
tensor([[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]])
```

In [28]:

```python

# 现在，让我们做一些稍微复杂的事情，在GPU上！
a = torch.ones(3, 4, device=cuda0, requires_grad=True)
b = torch.randn(4, 4, device=cuda0, requires_grad=True)

result = (torch.mm(a, b.t().exp()) * 0.5).sum() * b.prod() - b.mean()
print('this complicated chain of operation gives....')
print(result)
```

```
this complicated chain of operation gives....
tensor(-0.1125, device='cuda:0', grad_fn=<SubBackward0>)
```

In [29]:

```python
result.backward()
print('\ngradient wrt a is')
print(a.grad)
print('\ngradient wrt b is')
print(b.grad)
```

```
gradient wrt a is
tensor([[0.0002, 0.0002, 0.0002, 0.0007],
        [0.0002, 0.0002, 0.0002, 0.0007],
        [0.0002, 0.0002, 0.0002, 0.0007]], device='cuda:0')

gradient wrt b is
tensor([[-0.0713, -0.0537, -0.0754, -0.0568],
        [-0.0664,  0.0086, -0.0787, -0.0586],
        [-0.0586, -0.0664, -0.0538, -0.0591],
        [-0.0704, -0.0666, -0.0707, -0.0590]], device='cuda:0')
```

练习: 计算梯度

Compute

$$z = \log\left(\frac{1}{\sum_i \exp(a_i)} \sum_j b_j\right) - \sum_k b_k,$$

and then the gradients of z w.r.t. $\mathbf{a}$ and $\mathbf{b}$.

获此答案即可得分:

```
# Gradient wrt a
tensor([-0.0012, -0.0024, -0.0046, -0.0089, -0.0174, -0.0339, -0.0659, -0.1284,
        -0.2501, -0.4872])

# Gradient wrt b
tensor([-0.9963, -0.9963, -0.9963, -0.9963, -0.9963, -0.9963, -0.9963, -0.9963,
        -0.9963, -0.9963])
```

In [30]:

```
#########################
#                       #
#       Exercise        #
#                       #
#########################


a = torch.linspace(-3, 3, 10, dtype=torch.float32, requires_grad=True)
b = torch.logspace(0.2, 2, 10, requires_grad=True)


# ANSWER
# z = (a + b).sum() # 按照上式的要求修改这个函数，这将作为本次练习的得分点之一
z = torch.log(1.0 / torch.sum(torch.exp(a), dim=-1) * torch.sum(b, dim=-1)) - torch.sum(b,
print(z)

z.backward()

print(a.grad)
print(b.grad)
```

```
tensor(-266.3888, grad_fn=<SubBackward0>)
tensor([-0.0012, -0.0024, -0.0046, -0.0089, -0.0174, -0.0339, -0.0659, -0.1284,
        -0.2501, -0.4872])
tensor([-0.9963, -0.9963, -0.9963, -0.9963, -0.9963, -0.9963, -0.9963, -0.9963,
        -0.9963, -0.9963])
```

Manipulating the `requires_grad` flag

In [31]:

```

# 除了在创建张量时直接设置 `requires_grad` 标志外，你还可以使用 `my_tensor.requires_grad_()`

x = torch.randn(1, 4, 5)
print(x)
print('x does not track gradients')
```

```
tensor([[[-0.8213,  0.1622, -0.1784,  0.8579, -0.5715],
         [-0.0902, -0.1254, -1.2814,  1.6632,  1.4054],
         [-0.8403,  0.9673,  0.5899, -0.2480,  0.5982],
         [-0.6682,  0.4026,  0.4983,  0.5829,  1.5456]]])
x does not track gradients
```

In [32]:

```
x.requires_grad_()
print(x)
print('x now **does** track gradients')
```

```
tensor([[[-0.8213,  0.1622, -0.1784,  0.8579, -0.5715],
         [-0.0902, -0.1254, -1.2814,  1.6632,  1.4054],
         [-0.8403,  0.9673,  0.5899, -0.2480,  0.5982],
         [-0.6682,  0.4026,  0.4983,  0.5829,  1.5456]]], requires_grad=True)
x now **does** track gradients
```

常见误区

你不能对已标记 `requires_grad=True` 的变量进行原地操作，因为这会干扰 PyTorch 追踪梯度的能力。

In [33]:

```
try:
  # This will throw an error!
  x[0] = -1

except RuntimeError as e:
  print('Error:', e)

```

```
Error: a view of a leaf Variable that requires grad is being used in an in-place operation.
```

3. 灵活高效的神经网络库

`torch.nn` (https://pytorch.org/docs/stable/nn.html) 和 `torch.optim` (https://pytorch.org/docs/stable/optim.html) 这两个包提供了许多高效的神经网络组件实现，具体如下：

- 仿射层和激活函数 (https://pytorch.org/docs/stable/nn.html#non-linear-activations-weighted-sum-nonlinearity)
- Initialization schemes (https://pytorch.org/docs/stable/nn.html#torch-nn-init)
- Loss functions (https://pytorch.org/docs/stable/nn.html#loss-functions)
- Embeddings (https://pytorch.org/docs/stable/nn.html#sparse-layers)
- 多GPU训练 (https://pytorch.org/docs/stable/nn.html#dataparallel-layers-multi-gpu-distributed)
- 优化器 (https://pytorch.org/docs/stable/optim.html)
- 学习率调节 (https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate)
- etc.

In [34]:

```
import torch.nn as nn
import torch.nn.functional as F
```

In [35]:

```
#@title Visualization utility

# visualization utilities

import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib import rc
rc('animation', html='jshtml')

def plot_animation(model):
    # model should be on cuda0
    # assumes SGD optimization
    optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
    fig, ax = plt.subplots()
    num_frames = 100
    num_epochs = 5000
    epochs_per_frame = num_epochs // num_frames
    x = torch.randn(100, 8, device=cuda0)

    def animate(i):
        fig.clear()
        y = model(x)
        cpu_x = x.detach().cpu().numpy().flatten()
        cpu_y = y.detach().cpu().numpy().flatten()
        true_y = np.cos(cpu_x)
        perm = np.argsort(cpu_x)
        for j in range(i * epochs_per_frame, (i+1)*epochs_per_frame):
          train(model, optimizer, j)
        plt.plot(cpu_x[perm], cpu_y[perm])
        plt.plot(cpu_x[perm], true_y[perm], "-r")
        plt.xlim([-2, 2])
        plt.ylim([-2, 2])
        plt.draw()

    return animation.FuncAnimation(fig, animate,frames=num_frames, interval=10)
```

`torch.nn` 层

我们将以[全连接线性层（`nn.Linear`）(https://pytorch.org/docs/stable/nn.html#torch.nn.Linear)](https://pytorch.org/docs/stable/nn.html#torch.nn.Linear)为例进行说明。

全连接层会使用一个二维的权重参数 $\mathbf{w}$ 和一个一维的偏置参数 $\mathbf{b}$ 执行一次仿射变换：

$$f(\mathbf{x}) = \mathbf{w}^{\mathrm{T}}\mathbf{x} + \mathbf{b}.$$

In [36]:

```
fc = nn.Linear(in_features=8, out_features=8)
print(fc)
```

```
Linear(in_features=8, out_features=8, bias=True)
```

In [37]:

```
# It has two parameters, the weight and the bias

for name, p in fc.named_parameters():
    print('param name: {}\t shape: {}'.format(name, p.shape))
```

```
param name: weight       shape: torch.Size([8, 8])
param name: bias         shape: torch.Size([8])
```

In [38]:
```
fc.weight
```

Out[38]:
```
Parameter containing:
tensor([[-0.1746,  0.3516,  0.1566, -0.3313,  0.3460,  0.3418, -0.0801, -0.2676],
        [ 0.1961,  0.2718, -0.3518,  0.1791,  0.2271, -0.3315, -0.3287, -0.0863],
        [ 0.3001,  0.0260, -0.2076,  0.0579, -0.1870, -0.0445,  0.0365,  0.0532],
        [ 0.2905, -0.2907,  0.2016,  0.0432,  0.1254,  0.1089,  0.0406, -0.2203],
        [-0.3073, -0.1804,  0.2641, -0.3170, -0.1875, -0.1171,  0.1171, -0.2490],
        [-0.1056, -0.0247, -0.1607,  0.0167, -0.0363, -0.3527, -0.0143, -0.2232],
        [-0.0377, -0.1750, -0.3071,  0.0893, -0.2737,  0.2624,  0.2000,  0.2039],
        [ 0.3173,  0.0577, -0.3385, -0.1635,  0.3270,  0.1729,  0.0058,  0.0222]],
       requires_grad=True)
```

In [39]:
```
# 默认情况下，这些参数的 `requires_grad` 属性为 `True`，因此它们会收集梯度！

print(fc.bias)
```

```
Parameter containing:
tensor([ 0.2683,  0.2320, -0.2853, -0.2588,  0.0062, -0.1337, -0.3129,  0.1199],
       requires_grad=True)
```

In [40]:
```
# 让我们构建一个二维输入张量：
#  - 批量维度大小为 64
#  - 具有 8 个特征

x = torch.randn(64, 8)
```

In [41]:
```
# Pass it through the fc layer

result = fc(x)
print(result.shape)

# Why does the `result` have shape [64, 8]?
#   - batch dimension of size 64
#   - 8 output features
```

```
torch.Size([64, 8])
```

In [42]:
```
# 尽管输入 `x` 的 `requires_grad` 为 `False`，但网络的权重和偏置参数的 `requires_grad` 为 `Tr
print(result.requires_grad)
print(result.grad_fn)  # 它显示为 `AddmmBackward` 因为全连接层执行了矩阵乘法和加法操作。
```

```
True
<AddmmBackward0 object at 0x00000188A155BAC0>
```

In [43]:
```
# Say (arbitrarily) we want the layer to behave like the cosine function (yes I know it is ir

target = x.cos()
```

In [44]:
```
# Let's try MSE loss

loss = F.mse_loss(result, target)
print(loss)
```

```
tensor(1.0955, grad_fn=<MseLossBackward0>)
```

In [45]:
```
# Compute gradients

loss.backward()
print(fc.bias.grad)
```

```
tensor([-0.0600, -0.0632, -0.2243, -0.2339, -0.1662, -0.1755, -0.2281, -0.1008])
```

In [46]:
```
#手动演示梯度下降

print('bias before GD', fc.bias)

lr = 0.5
with torch.no_grad():
    # this context manager tells PyTorch that we don't want ops inside to be
    # tracked by autograd!
    for p in fc.parameters():
        p -= lr * p.grad

print('bias after GD', fc.bias)
```

```
bias before GD Parameter containing:
tensor([ 0.2683,  0.2320, -0.2853, -0.2588,  0.0062, -0.1337, -0.3129,  0.1199],
       requires_grad=True)
bias after GD Parameter containing:
tensor([ 0.2983,  0.2636, -0.1731, -0.1418,  0.0893, -0.0459, -0.1989,  0.1703],
       requires_grad=True)
```

`torch.optim` 优化器

更简便的做法是，我们可以使用 PyTorch 提供的 `torch.optim` (https://pytorch.org/docs/stable/optim.html#torch.optim) 优化器。下面让我们以 `torch.optim.SGD` (https://pytorch.org/docs/stable/optim.html#torch.optim.SGD) 优化器为例进行说明！

In [47]:
```
# Let's optimize for 5000 iterations

# First, put the layer on GPU so things run faster

fc = fc.to(cuda0)
```

In [48]:
```
# Construct an optimizer

optim = torch.optim.SGD(fc.parameters(), lr=0.1)
```

In [49]:

```
batch_size = 256

# training loop (refactored as function to be used in visualization)
def train(model, optimizer, idx):
    # clear gradients accumulated on the parameters
    optimizer.zero_grad()

    # get an input (say we only care inputs sampled from N(0, I))
    x = torch.randn(batch_size, 8, device=cuda0)  # this has to be on GPU too

    # target is the cos(x)
    target = torch.cos(x)

    # forward pass
    result = model(x)

    # compute loss
    loss = F.mse_loss(result, target)

    # compute gradients
    loss.backward()

    # let the optimizer do its work; the parameters will be updated in this call
    optimizer.step()

    # add some printing
    if idx % 500 == 0:
        print('iteration {}\tloss {:.5f}'.format(idx, loss))

for i in range(5000):
    train(fc, optim, i)
```

```
iteration 0     loss 0.84597
iteration 500   loss 0.20310
iteration 1000  loss 0.21011
iteration 1500  loss 0.21311
iteration 2000  loss 0.19416
iteration 2500  loss 0.20789
iteration 3000  loss 0.19231
iteration 3500  loss 0.19428
iteration 4000  loss 0.18239
iteration 4500  loss 0.20162
```

In [50]:

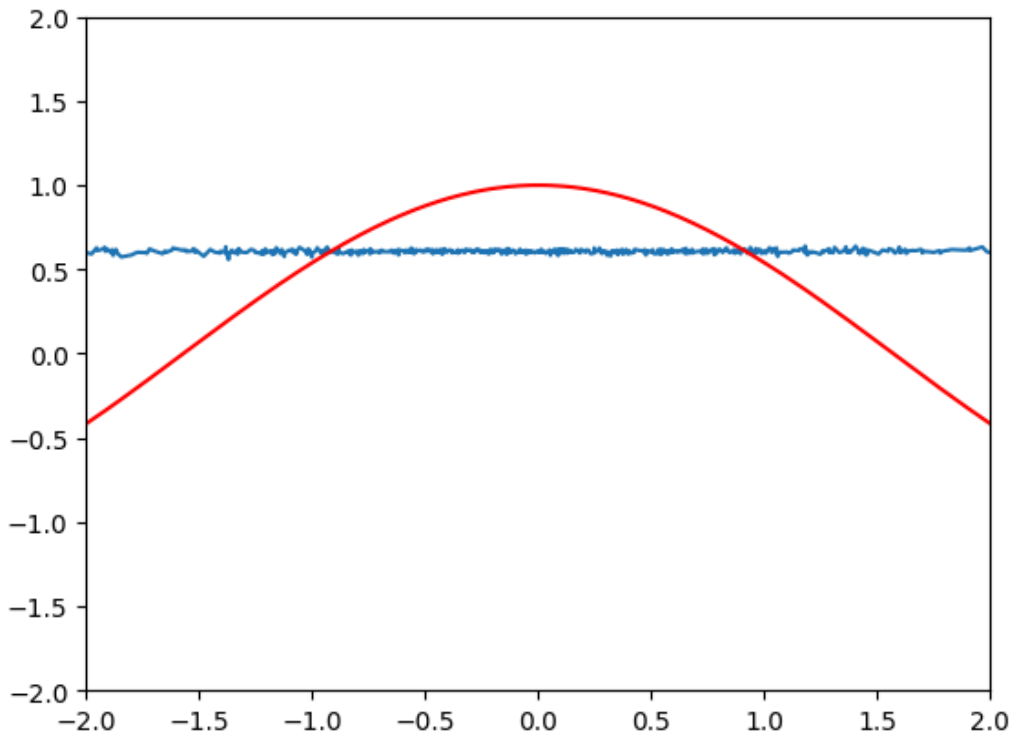
```
# re-initialize model
fc = nn.Linear(in_features=8, out_features=8).to(cuda0)

# visualization
plot_animation(fc)
```

```
iteration 0     loss 1.16435
iteration 0     loss 0.28239
iteration 500   loss 0.19080
iteration 1000  loss 0.20155
iteration 1500  loss 0.21363
iteration 2000  loss 0.18267
iteration 2500  loss 0.21902
iteration 3000  loss 0.19651
iteration 3500  loss 0.21412
iteration 4000  loss 0.21081
iteration 4500  loss 0.18936
```

Out[50]:

Once Loop Reflect

```
iteration 0     loss 0.21423
```



构建深度神经网络

单一的 `nn.Linear` 层表现欠佳！上述均方误差（MSE）损失仍然相当大。

不过这在意料之中，因为它仅仅是一个线性变换，所以表达能力有限。让我们用一个深度网络来替代它，看看效果如何！

为了简单起见，我们将使用以下前馈网络架构（从上到下）：

```
        [输入]
          ||
[全连接层 8 -> 32]
          ||
    [ReLU 激活函数]
          ||
[全连接层 32 -> 32]
          ||
    [ReLU 激活函数]
```

在 PyTorch 中，一个模型由一个 `nn.Module` (https://pytorch.org/docs/stable/nn.html#torch.nn.Module) 对象来表示。我们上面所探讨的 `nn.Linear` 层同样也是该类的一个实例：

现在我们想要构建一个深度网络，我们可以通过自己编写一个自定义的 nn.Module 类，将所需的层组合在一起。

In [51]:

```python
class MyNet(nn.Module):  # subclass nn.Module
    def __init__(self):
        super(MyNet, self).__init__()

        # We need 3 fully-connected layers!
        # Simply assigning them as attributes will
        # make sure that PyTorch keeps track of them.

        # 8 => 32
        self.fc1 = nn.Linear(8, 32)
        # 32 => 32
        self.fc2 = nn.Linear(32, 32)
        # 32 => 8
        self.fc3 = nn.Linear(32, 8)


    # We also need to define a `forward()` method that details
    # what should happen when this module is used.
    def forward(self, x):
        x = self.fc1(x)
        x = x.relu()
        x = self.fc2(x)
        x = x.relu()
        return self.fc3(x)
```

In [52]:

```python
# Okay! Now we are ready to use this deep network!

# Construct a network and move to GPU
net = MyNet().to(cuda0)

# Construct an optimizer
optim = torch.optim.SGD(net.parameters(), lr=0.1)
```

In [53]:

```
# The same training loop, but now using a deep network!

batch_size = 256

for i in range(5000):
    train(net, optim, i)
```

```
iteration 0     loss 0.56846
iteration 500   loss 0.15856
iteration 1000  loss 0.09475
iteration 1500  loss 0.03602
iteration 2000  loss 0.02295
iteration 2500  loss 0.01500
iteration 3000  loss 0.00938
iteration 3500  loss 0.00936
iteration 4000  loss 0.00797
iteration 4500  loss 0.00809
```

The network did so much better than a single fully-connected layer!

In [54]:

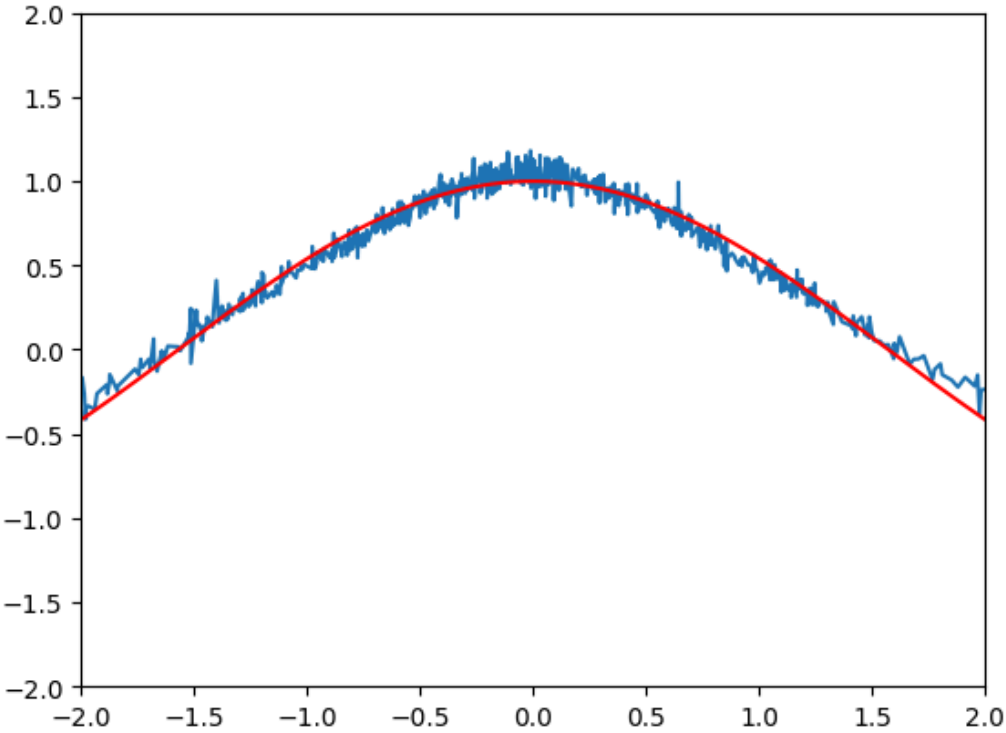
```
# re-initialize model
net = MyNet().to(cuda0)

# visualization
plot_animation(net)
```

```
iteration 0     loss 0.59914
iteration 0     loss 0.23558
iteration 500   loss 0.17753
iteration 1000  loss 0.08883
iteration 1500  loss 0.03376
iteration 2000  loss 0.01160
iteration 2500  loss 0.00894
iteration 3000  loss 0.00804
iteration 3500  loss 0.00737
iteration 4000  loss 0.00590
iteration 4500  loss 0.00527
```

Out[54]:

Once Loop Reflect

```
iteration 0     loss 0.00562
```



更多 nn.* 层

nn.* 包中还提供了许多其他的层。下面列举几个：

- 卷积层：例如， nn.Conv2d (https://pytorch.org/docs/stable/nn.html#torch.nn.Conv2d)
- 归一化层：例如， nn.BatchNorm2d (https://pytorch.org/docs/stable/nn.html#torch.nn.BatchNorm2d)
- 激活函数层：例如， nn.ReLU (https://pytorch.org/docs/stable/nn.html#torch.nn.ReLU)
- 等等。

In [55]:
```
# A conv 2d layer with 4x4 filters, mapping inputs with 3 channels to outputs with 5 channels

conv = nn.Conv2d(in_channels=3, out_channels=5, kernel_size=4)
```

In [56]:
```
# It also has two parameters, the weight and the bias

for name, p in conv.named_parameters():
    print('param name: {}\t shape: {}'.format(name, p.shape))

# Why does the weight have shape [5, 3, 4, 4]?
```

```
param name: weight       shape: torch.Size([5, 3, 4, 4])
param name: bias         shape: torch.Size([5])
```

`nn.Module` **容器**

`torch.nn` 还提供了许多其他的 `nn.Module` 容器 (https://pytorch.org/docs/stable/nn.html#containers)，用于轻松构建复杂的网络。例如， `nn.Sequential` (https://pytorch.org/docs/stable/nn.html#torch.nn.Sequential) 会按顺序执行一系列子模块，将每个模块的输出作为下一个模块的输入。

使用 `nn.Sequential` ，上述网络可以等效地写成:

In [57]:
```
net = nn.Sequential(
    nn.Linear(8, 32),
    nn.ReLU(),                # This nn.Module does the ReLU activation on its input
    nn.Linear(32, 32),
    nn.ReLU(),
    nn.Linear(32, 8),
).to(cuda0)
```

4. （实践）构建一个 MNIST 数字分类器

利用我们所学的知识，让我们来构建一个简单的 MNIST 手写数字分类器。

In [58]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F


torch.manual_seed(1234)  # for reproducibility

cuda0 = torch.device('cuda:0')
```

加载数据

`torchvision` (https://pytorch.org/docs/stable/torchvision/index.html) 库提供了广泛的标准视觉数据集，以及带有预训练权重的网络。我们将使用 `torchvision.datasets.MNIST` 类 (https://pytorch.org/docs/stable/torchvision/datasets.html#mnist) 来方便地访问 MNIST 数据集。你也可以编写自己的自定义数据集 (https://pytorch.org/tutorials/beginner/data_loading_tutorial.html)。

In [59]:
```
import torchvision
# load MNIST dataset
# mnist_train = torchvision.datasets.MNIST(root='./data', download=True, train=True)
mnist_train = torchvision.datasets.MNIST(root='./data', download=False, train=True)
```

Each element of the dataset is a 2-tuple, consisting of the image of the digit, and its label. E.g.,

In [60]:
```
image, label = mnist_train[13]
print('This is a digit {}:'.format(label))
image
```

```
This is a digit 6:
```

Out[60]:

In [61]:
```
type(image)
```

Out[61]:
```
PIL.Image.Image
```

图像是以 `PIL.Image` 格式给出的。为了自动获得一个 `torch.Tensor` ，我们可以在构建数据集时添加一个 `torchvision.transforms.ToTensor` (https://pytorch.org/docs/stable/torchvision/transforms.html#torchvision.transforms.ToTensor) 变换：

In [62]:
```
mnist_train = torchvision.datasets.MNIST(root='./data', download=False, train=True,
                                          transform=torchvision.transforms.ToTensor())  # the
```

In [63]:
```
image, label = mnist_train[13]
print(type(image))
print(image.shape)  # MNIST images are 28x28, and have a single channel representing brightne
```

```
<class 'torch.Tensor'>
torch.Size([1, 28, 28])
```

In [64]:
```
print('This image has max={}'.format(image.max()), 'and min={}'.format(image.min()))
```

```
This image has max=1.0 and min=0.0
```

在深度学习中，将网络输入进行归一化处理，使其均值为零，通常是个不错的做法。我们可以使用 `torchvision.transforms.Normalize` (https://pytorch.org/docs/stable/torchvision/transforms.html#torchvision.transforms.Normalize)变换来实现这一点。添加这个变换后，我们可以按如下方式构建数据集：

In [65]:
```
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize(mean=(0.5,), std=(0.5,)),  # [0, 1] range => [-1, 1] ra
])

mnist_train = torchvision.datasets.MNIST(root='./data', download=False, train=True,
                                          transform=transform)
mnist_val = torchvision.datasets.MNIST(root='./data', download=False, train=False,
                                          transform=transform)
```

In [66]:
```
image, label = mnist_train[13]
print(type(image))
print(image.shape)  # MNIST images are 28x28, and have a single channel representing brightne
```

```
<class 'torch.Tensor'>
torch.Size([1, 28, 28])
```

In [67]:
```
print('This image has max={}'.format(image.max()), 'and min={}'.format(image.min()))
```

```
This image has max=1.0 and min=-1.0
```

In [68]:
```
print('training set size:\t{}'.format(len(mnist_train)))
print('validation set size:\t{}'.format(len(mnist_val)))
```

```
training set size:      60000
validation set size:    10000
```

我们使用 PyTorch 的 `torch.utils.data.DataLoader` 来自动加载批量数据。

In [69]:
```
batch_size = 512

train_loader = torch.utils.data.DataLoader(mnist_train,
                                           batch_size=batch_size,
                                           shuffle=True,                    # shuffle trainin
                                           num_workers=4,                  # turns on multi-p
                                           pin_memory=True)                # pin_memory allo
val_loader = torch.utils.data.DataLoader(mnist_val,
                                         batch_size=batch_size,
                                         num_workers=4,
                                         pin_memory=True)
```

In [70]:
```
# `train_loader`（一个 Python 可迭代对象）产生的每个元素仍然是一个二元元组，但现在这个元组由

images, labels = next(iter(train_loader))
print('batched image tensor shape: {}'.format(images.shape))
print('batched label tensor shape: {}'.format(labels.shape))
```

```
batched image tensor shape: torch.Size([512, 1, 28, 28])
batched label tensor shape: torch.Size([512])
```

构建网络

我们将使用一个卷积网络进行分类。以下架构改编自著名的 [LeNet - 5 (https://ieeexplore.ieee.org/document/726791)](https://ieeexplore.ieee.org/document/726791) [1]。

[1] LeCun, Yann, et al. "Gradient-based learning applied to document recognition." Proceedings of the IEEE 86.11 (1998): 2278-2324.

In [71]:
```
class MyNet(nn.Module):
    def __init__(self):
        super(MyNet, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, padding=2)
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5)
        self.fc1 = nn.Linear(in_features=16 * 5 * 5, out_features=120)
        self.fc2 = nn.Linear(in_features=120, out_features=84)
        self.fc3 = nn.Linear(in_features=84, out_features=10)

    def forward(self, x):
        out = F.relu(self.conv1(x))
        out = F.max_pool2d(out, 2)
        out = F.relu(self.conv2(out))
        out = F.max_pool2d(out, 2)
        out = out.reshape(out.size(0), -1)
        out = F.relu(self.fc1(out))
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

In [72]:
```
net = MyNet().to(cuda0)
```

In [73]:
```
# This network output a size 10 vector for each input image, as verified below
# using a random input tensor.

random_input = torch.randn(123, 1, 28, 28, device=cuda0)
output = net(random_input)
print(output.shape)
```

```
torch.Size([123, 10])
```

训练循环

对于分类任务，我们将使用交叉熵损失函数 `F.cross_entropy` (https://pytorch.org/docs/stable/nn.html#torch.nn.functional.cross_entropy) 来训练这个网络。

我们可以编写一个函数，通过 `train_loader` 遍历训练集并完成一个轮次（epoch）的训练。这段代码与我们之前近似 $f(x) = \cos(x)$ 时所写的代码极为相似，只是在数据加载和损失计算的方式上有所不同。

In [74]:

```
def train(net, optim):
    net.train()
    for image_cpu, label_cpu in train_loader:
        # put data onto GPU
        image = image_cpu.to(cuda0)
        label = label_cpu.to(cuda0)

        # clear gradient
        optim.zero_grad()

        # forward through the network
        output = net(image)

        # compute loss and gradient
        loss = F.cross_entropy(output, label)
        loss.backward()

        # update parameters
        optim.step()
```

让我们再编写一个函数，用于在验证集上评估我们的网络。

In [75]:

```
def evaluate(net):
    total = 0
    correct = 0

    # 将网络设置为评估模式。当网络中存在在训练和评估阶段行为不同的层时，这一步很重要，
    # 例如，Dropout 和 Batch Normalization 层。
    net.eval()
    for image_cpu, label_cpu in val_loader:
        # 将数据移动到 GPU 上
        image = image_cpu.to(cuda0)
        label = label_cpu.to(cuda0)

        # 在这个上下文管理器中不跟踪梯度
        # 因为我们正在进行评估，不需要梯度，这样可以节省一些时间和 GPU 内存。
        with torch.no_grad():
            # 前向传播通过网络，并获取预测的类别
            prediction = net(image).argmax(dim=-1)

            total += image.size(0)  # 批量大小
            # `.item()` 从单元素张量中提取一个 Python（cpu） 数值
            correct += (prediction == label).sum().item()

    return correct / total
```

In [76]:

```
# Without any training, the network accuracy matches that of random guessing: ~10%.

print('At initialization, the network has accuracy {:.4f}%'.format(evaluate(net) * 100))
```

```
At initialization, the network has accuracy 10.7200%
```

Putting Everything Together

In [77]:

```
num_epochs = 10
lr = 0.1

optim = torch.optim.SGD(net.parameters(), lr=lr)

for epoch in range(num_epochs):
    print('Epoch: {}\tValidation Accuracy: {:.4f}%'.format(epoch, evaluate(net) * 100))
    train(net, optim)

print('Done! \tValidation Accuracy: {:.4f}%'.format(evaluate(net) * 100))
```

```
Epoch: 0        Validation Accuracy: 10.7200%
Epoch: 1        Validation Accuracy: 79.1900%
Epoch: 2        Validation Accuracy: 92.1300%
Epoch: 3        Validation Accuracy: 95.3100%
Epoch: 4        Validation Accuracy: 95.8500%
Epoch: 5        Validation Accuracy: 97.7700%
Epoch: 6        Validation Accuracy: 98.0600%
Epoch: 7        Validation Accuracy: 88.2100%
Epoch: 8        Validation Accuracy: 98.3300%
Epoch: 9        Validation Accuracy: 98.1700%
Done!   Validation Accuracy: 98.7200%
```

In [78]:

```
import PIL

# Utility function
def tensor2image(tensor):
    return PIL.Image.fromarray((tensor.detach() * 127.5 + 128).clamp_(0, 255).to('cpu', tor
```

In [79]:

```
# Get a data sample we want to attack

data, label = mnist_val[30]
data = data.to(cuda0)
```

In [80]:

```
# Look at the image

tensor2image(data)
```

Out[80]:

In [81]:

```
# Label is...

print('label is', label)
```

```
label is 3
```

In [82]:

```
# Let's see what our trained network says about this image.


# turn the network into eval mode
net.eval()
output = net(data.unsqueeze(0))  # .unsqueeze(0) insert a batch dimension so it looks like ba
prediction = output.argmax(-1)
print('predicted class is', prediction.item())
```

```
predicted class is 3
```

check more samples:

In [83]:

```
import matplotlib.pyplot as plt
for i in range(5):
    # Get a data sample we want to attack

    data, label = mnist_val[i]
    data = data.to(cuda0)
    # Look at the image

    plt.figure(figsize=(1,1))
    plt.imshow(tensor2image(data), cmap='gray')
    plt.axis('off')
    plt.show()

    # turn the network into eval mode
    net.eval()
    output = net(data.unsqueeze(0))  # .unsqueeze(0) insert a batch dimension so it looks lil
    prediction = output.argmax(-1)
    print('predicted class is', prediction.item())
```

```
predicted class is 7
```

```
predicted class is 2
```

```
predicted class is 1
```

```
predicted class is 0
```

```
predicted class is 4
```

恭喜你！你完成了 PyTorch 教程的学习！

作业: Build your own deeper net

执行相同的回归任务（即对 $f(x) = \cos(x)$ 进行建模），但进行以下修改：

- 多使用一个隐藏层
- 每个隐藏层应包含 128 个神经元
- 使用 `tanh` 激活函数（请参阅 `my_tensor.tanh()` (https://pytorch.org/docs/stable/tensors.html#torch.Tensor.tanh)）
- 使用 128 的批量大小
- 使用 `torch.optim.Adam` (https://pytorch.org/docs/stable/optim.html#torch.optim.Adam) 优化器
- 使用 L1 损失函数 (https://pytorch.org/docs/stable/nn.html#torch.nn.functional.l1_loss)

In [40]:

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib import rc
rc('animation', html='jshtml')

```

In [41]:

```

def train(model, optimizer, criterian, idx, device):
    optimizer.zero_grad()

    x = torch.randn(batch_size, 8, device=device)

    target = torch.cos(x)

    result = model(x)

    loss = criterian(result, target)
    loss.backward()

    optimizer.step()

    if idx % 500 == 0:
        print('iteration {}\tloss {:.5f}'.format(idx, loss))

```

In [46]:

```
def plot_animation(model, optimizer, criterian, device):
    fig, ax = plt.subplots()
    num_frames = 100
    num_epochs = 5000
    epochs_per_frame = num_epochs // num_frames
    x = torch.randn(100, 8, device=device)

    def animate(i):
        fig.clear()
        y = model(x)
        cpu_x = x.detach().cpu().numpy().flatten()
        cpu_y = y.detach().cpu().numpy().flatten()
        true_y = np.cos(cpu_x)
        perm = np.argsort(cpu_x)
        for j in range(i * epochs_per_frame, (i+1)*epochs_per_frame):
          train(model, optimizer, criterian, j, device)
        plt.plot(cpu_x[perm], cpu_y[perm])
        plt.plot(cpu_x[perm], true_y[perm], "-r")
        plt.xlim([-2, 2])
        plt.ylim([-2, 2])
        plt.draw()

    return animation.FuncAnimation(fig, animate,frames=num_frames, interval=10)
```

In [51]:

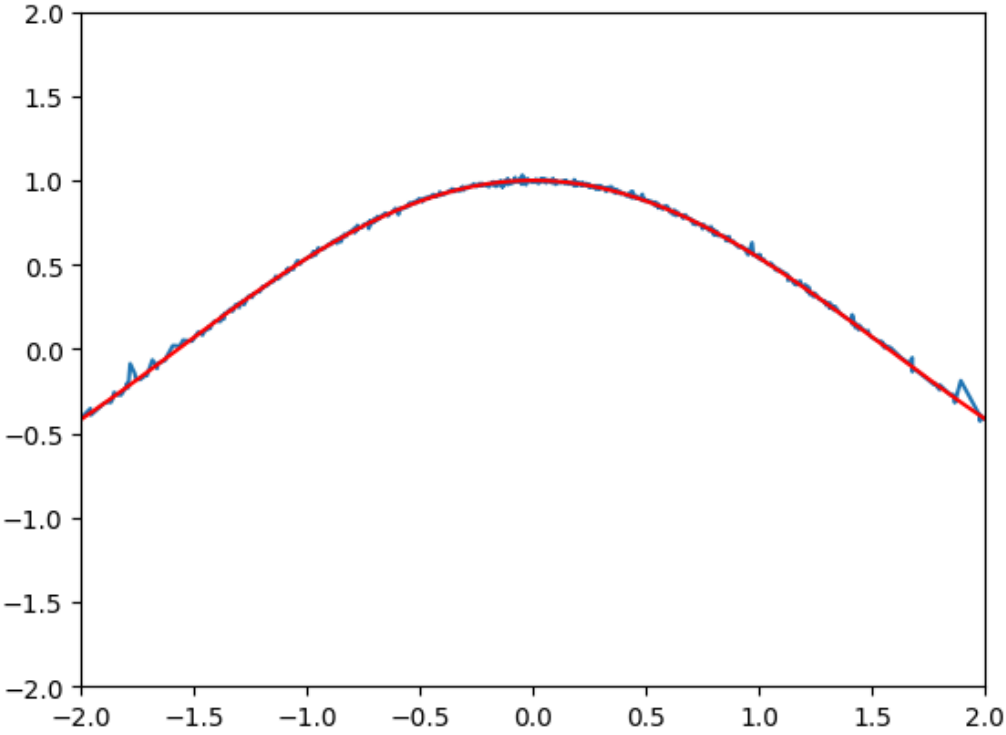
```
#########################
#                       #
#       Exercise        #
#                       #
#########################

# Answer
class MyNet2(nn.Module):
    def __init__(self):
        super(MyNet2, self).__init__()
        self.fc1 = nn.Linear(8, 128)
        self.fc2 = nn.Linear(128, 128)
        self.fc3 = nn.Linear(128, 128)
        self.fc4 = nn.Linear(128, 8)
        self.tanh = nn.Tanh()  # 可使用nn.Tanh激活函数

        self.net = nn.Sequential(
            nn.Linear(8, 128),
            nn.Tanh(),
            nn.Linear(128, 128),
            nn.Tanh(),
            nn.Linear(128, 128),
            nn.Tanh(),
            nn.Linear(128, 8)
        )

    def forward(self, x):
        return self.net(x)

device = 'cuda' if torch.cuda.is_available() else 'cpu'
lr = 3e-4
model = MyNet2().to(device)
optim = torch.optim.Adam(model.parameters(), lr=lr)
criterian = nn.L1Loss()
batch_size = 128

plot_animation(model, optim, criterian, device)

```

```
iteration 0     loss 0.68905
iteration 0     loss 0.33260
iteration 500   loss 0.07862
iteration 1000  loss 0.03312
iteration 1500  loss 0.02314
iteration 2000  loss 0.01505
iteration 2500  loss 0.01495
iteration 3000  loss 0.01367
iteration 3500  loss 0.01320
iteration 4000  loss 0.01128
iteration 4500  loss 0.00894
```

Out[51]:

Once Loop Reflect

```
iteration 0     loss 0.00938
```


2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0